

B.TECH. SEM IV

Course- Software Architecture Analysis

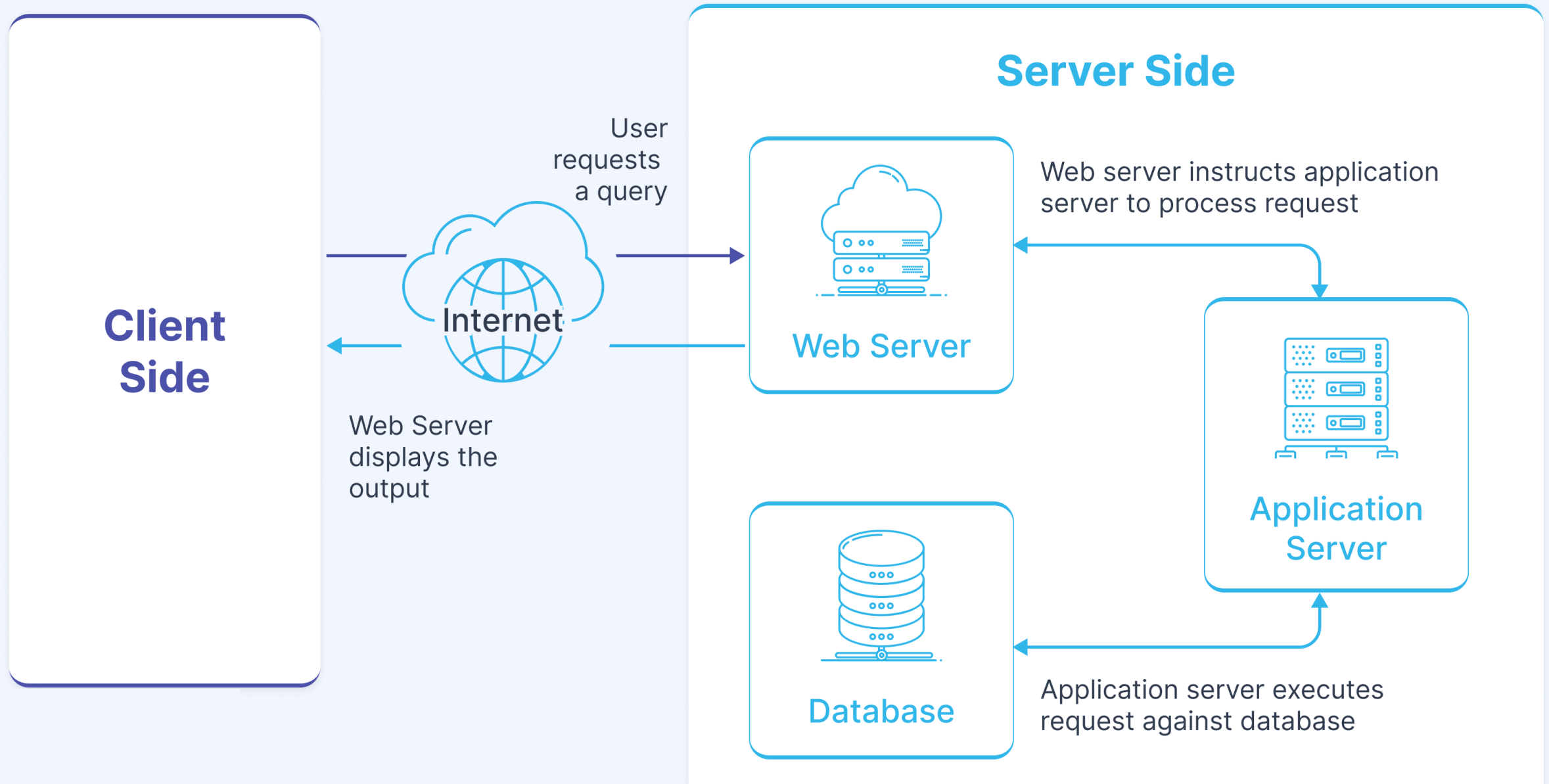
Course Code- 24CS02HT0401

Category- Honors

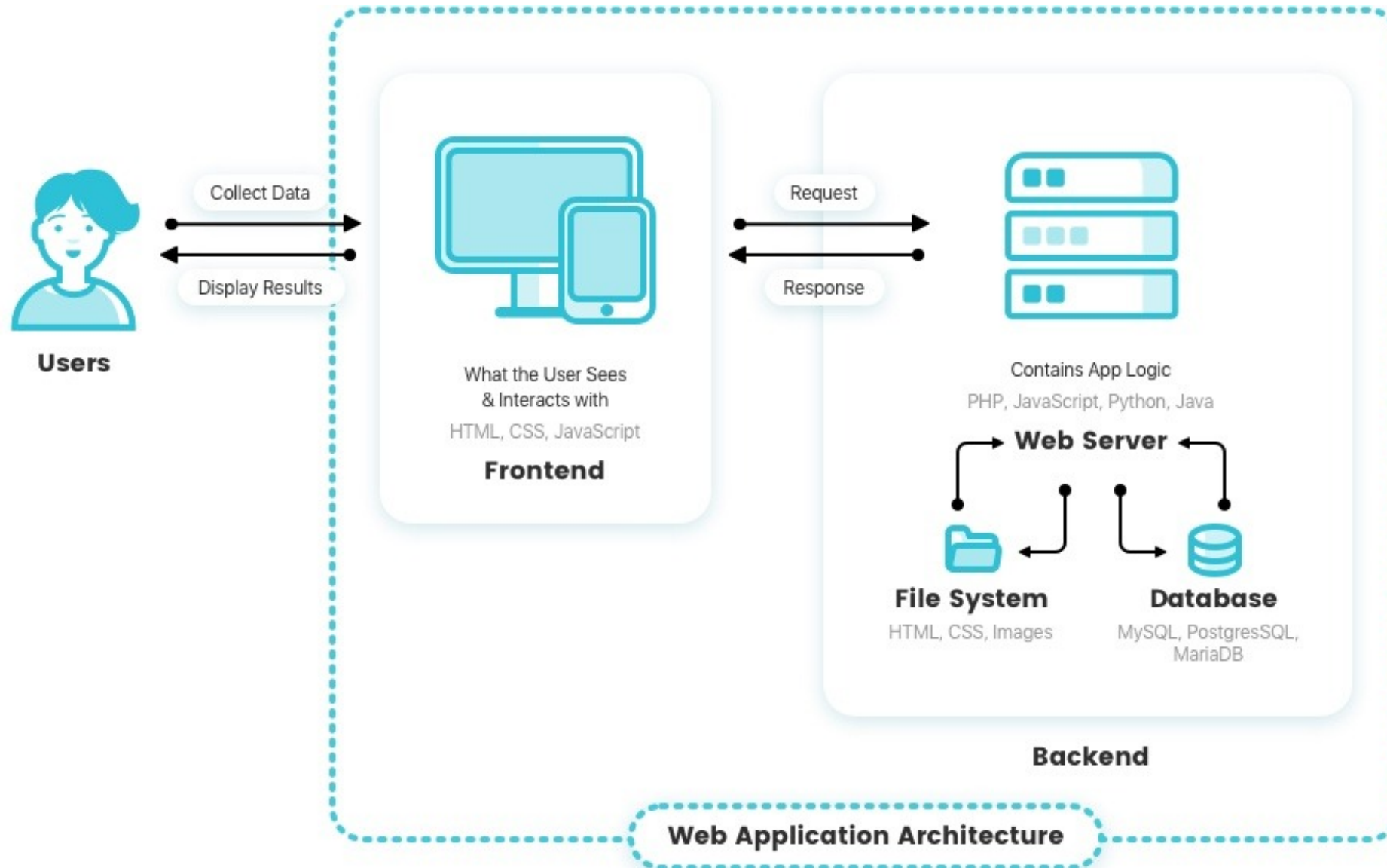


By,
Vaishali Katkar

How Web Application Works?

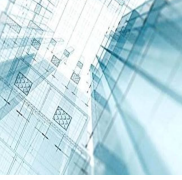


Web Application Architecture





Software Database Design Case Study



Develop an ordering system for Pizza Dash, a popular fast-food chain spread across the UK with approximately 250 restaurants.



Pizza Dash envisions an application that:

- Facilitates customers in placing and tracking their orders.
- Manages the order journey from kitchen preparation to delivery via various delivery partners.
- Serves customers from any location in the UK.
- **Features:**
 - Secure payment processing.
 - Real-time order tracking.
 - Customer recognition with insights into past orders and recommendations.
 - Storing order data for business analytics and future reference.

• Ordering Process

- 1 — A customer places an order via the mobile app or website.
- 2 — The order undergoes:
 - Payment processing through a gateway.
 - Asynchronous acknowledgement from the designated restaurant.
 - Continuous order tracking.
 - Transfer to a partnered delivery agency for final delivery.

• System Capabilities

- Withstand high traffic during rush hours without order loss.
- Recognise repeat customers and provide pertinent suggestions.
- Offer real-time updates at each stage of the order.
- Ensure data quality and integrity.

• Key Pillars

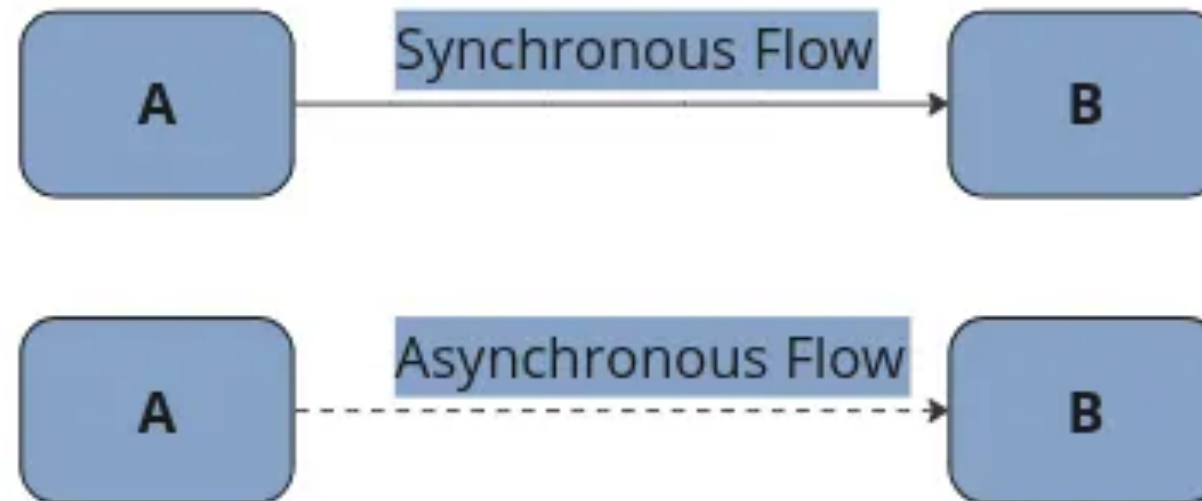
- Data consistency and security.
- Scalability.
- High availability & Load management.

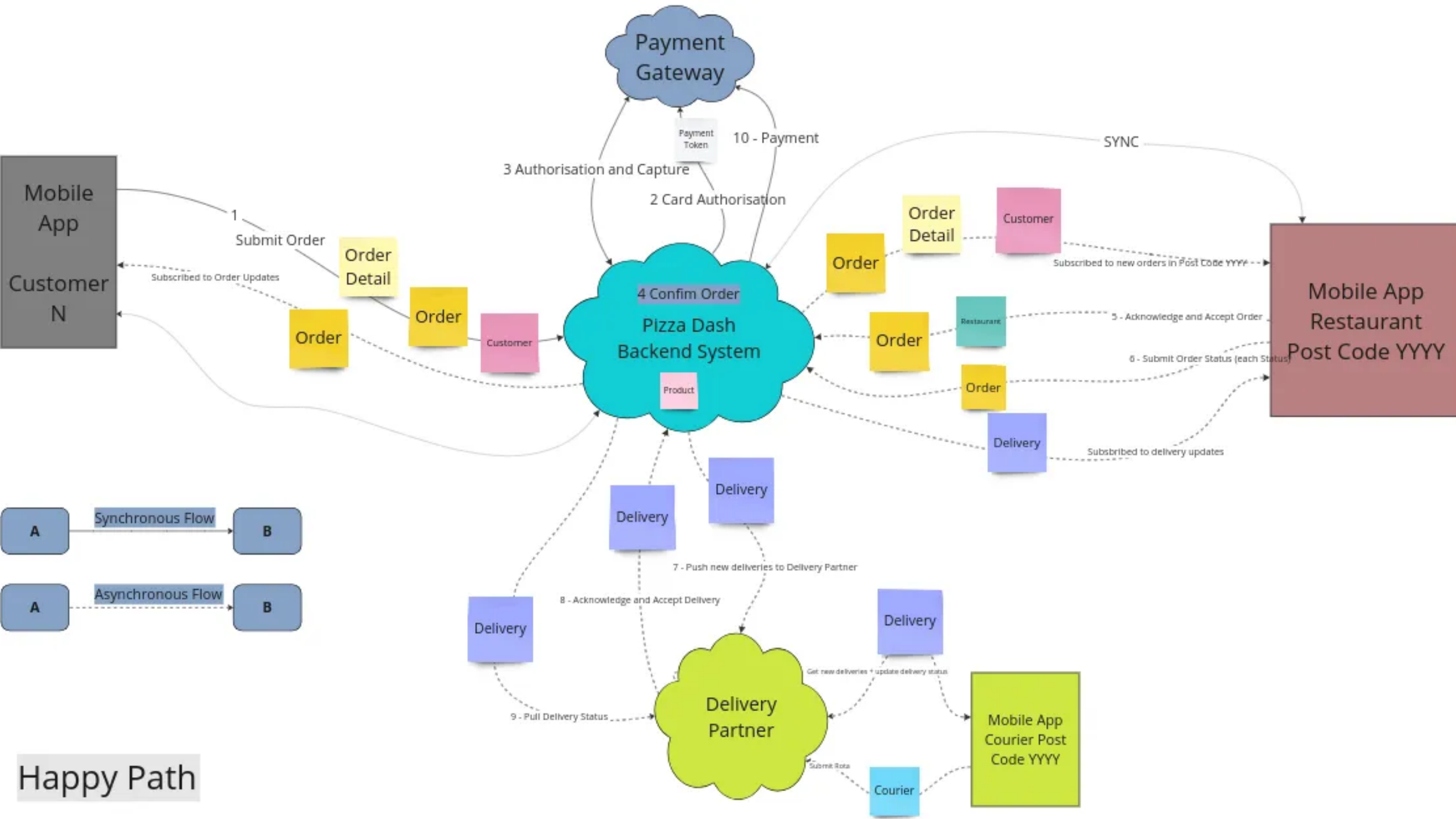
Task Overview

The assignment is to conceptualise an architectural solution meeting these requirements. Furnish a high-level architectural diagram and elucidate the data flows and journey within the proposed architecture.

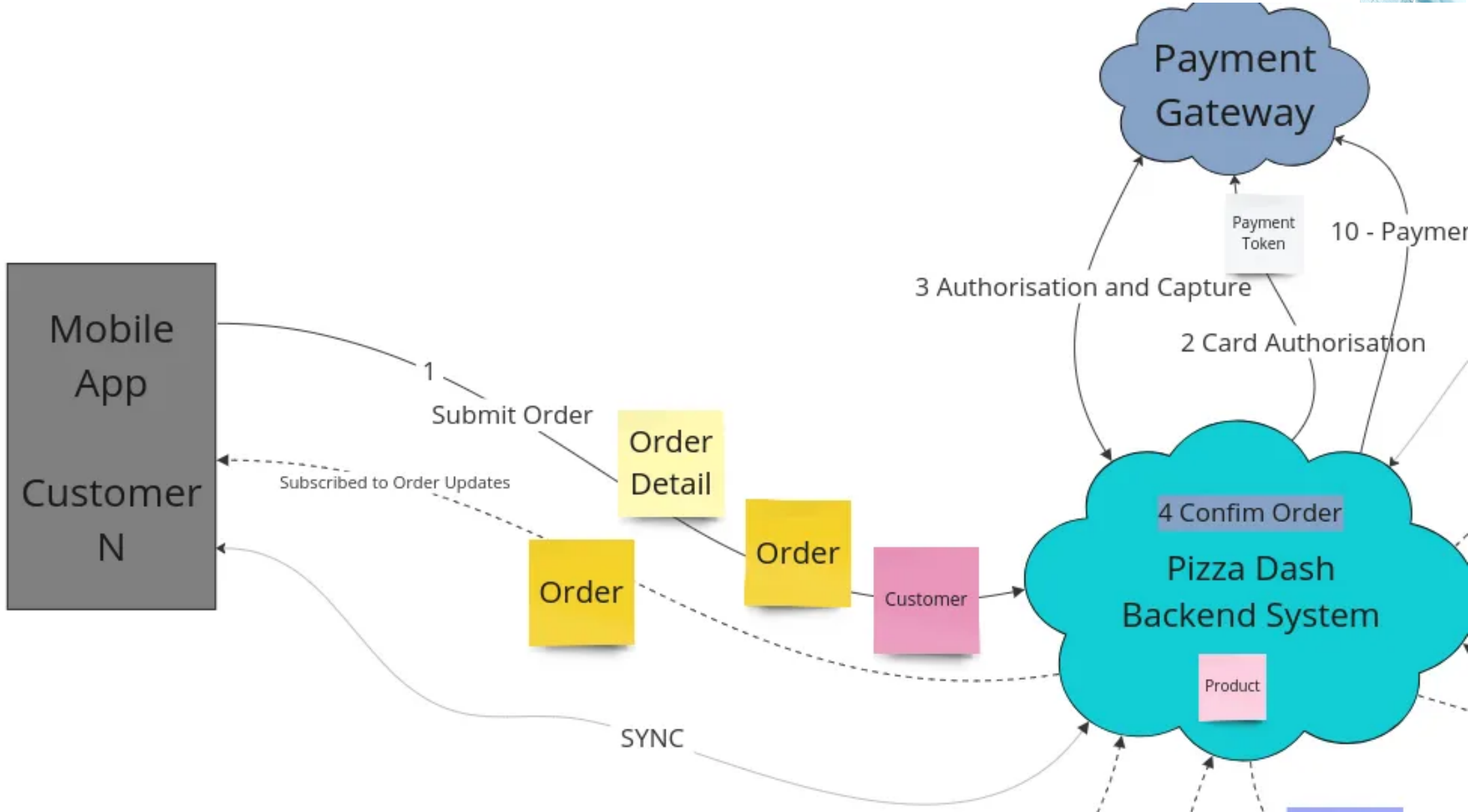
Proposed Solution

- The Pizza Dash Ordering System encompasses a mobile app and the central-backend supporting it. The diagram below shows the high level data flows that will be explained in detail. This will cover the Happy Path flow (nothing fails, every resource is available, no exceptions are found).
- We distinguish between two kinds of data flows: synchronous and asynchronous. The synchronous flow involves real-time data integration, usually characterised by point-to-point integration. On the other hand, the asynchronous flow is often associated with a publish/subscribe (pub/sub) messaging integration.





Happy Path





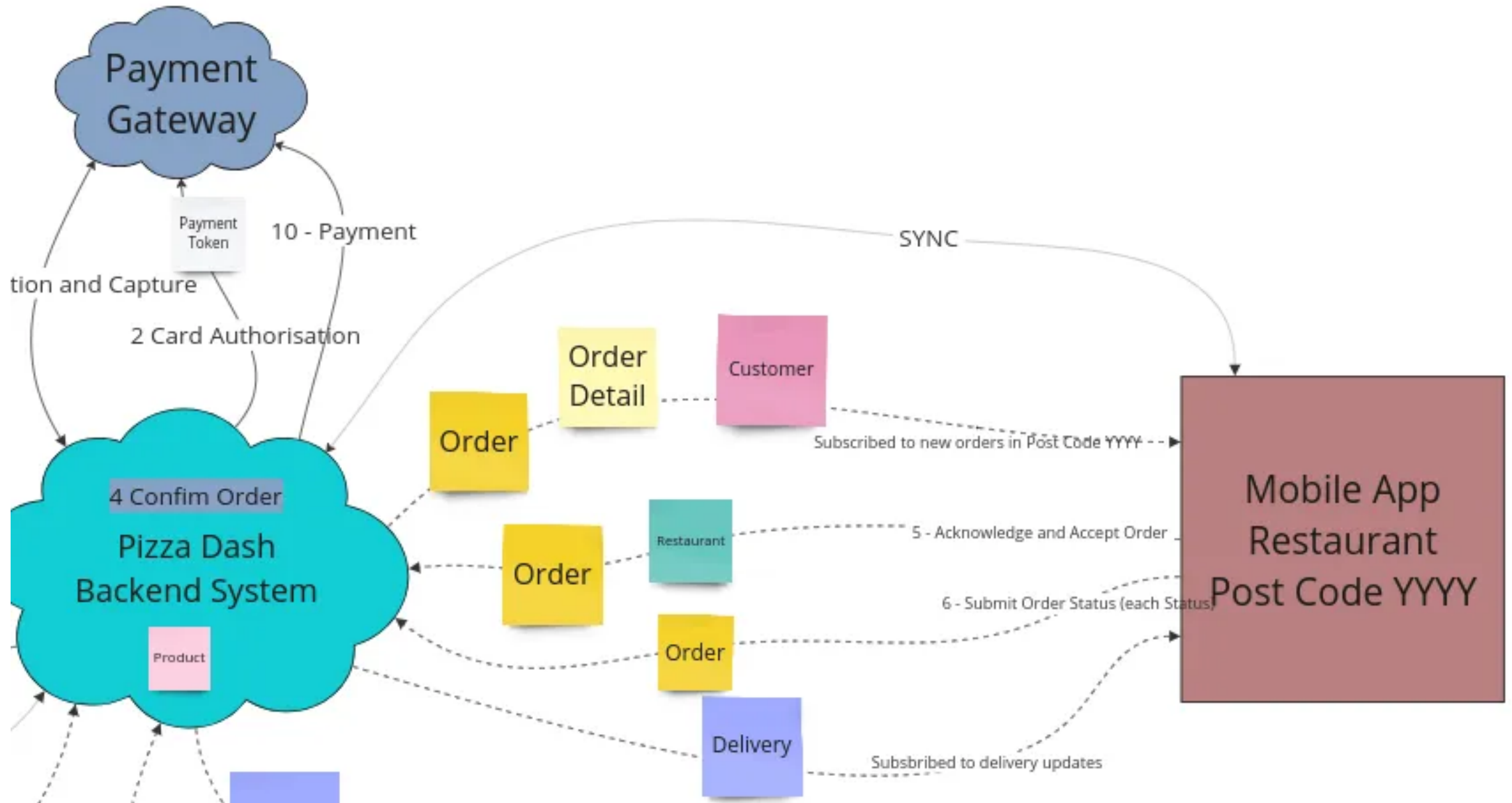
1 —When a customer places an order via the mobile app, it involves three data entities in a synchronous data flow: Customer, Order, and Order Detail (refer to the data model). This real-time integration with the backend system typically communicates using a JSON message payload with the following structure:



```
{
  "customer_order": {
    "customer": [
      {
        "customer_num": "1",
        "first_name": "Peter",
        "last_name": "Parker",
        "email": "p.parker@gmailo.com",
        "phone_num": "4422522289988",
        "address": "11 Parker Street KK001",
        "order": [
          {
            "order_num": "100050223",
            "order_datetime": "15-02-2026 14:02:01",
            "order_status": "Submitted",
            "payment_method_details": "payment_method_data",
            "order_detail": [
              {
```

```
                "order_det_num": "1",
                "product_num": "10002",
                "price product_qty": "2,99",
                "product_qty": 3
              },
              {
                "order_det_num": "2",
                "product_num": "10005",
                "price product_qty": "7,99",
                "product_qty": 2
              }
            ]
          }
        ]
      }
    ]
  }
}
```

- 2 — Upon reaching the backend, the system invokes the payment gateway to authorise the payment card. (Please note: For simplicity's, payment method details like the payment card within the order data entity. In a more detailed scenario, a distinct 'payment method' data entity would be required.)
- 3 — After authorisation, the payment is secured but not yet deducted. It's essential not to finalise the charge until the restaurant acknowledge and accept the order.
- 4 — Upon securing the payment, the order's status updates to "Payment Authorised". The restaurant, however, has not yet confirmed the order at this point.
- 5 — Restaurants are set up to receive notifications for new "Payment Authorised" orders intended for delivery within their pre-defined zones. As these orders arrive, the restaurant must acknowledge and approve them. Once that's done, the order status transitions to "Order Accepted".
- 6 — The restaurant then broadcasts various order status updates, which can be accessed in real-time by both the customer and the delivery partner.



- As the restaurant processes the order, its status gets updated according to the progress made.
- Once it's nearing completion, the status transitions to "Ready to Deliver".
- This status might be initiated a few minutes before the order is genuinely ready, ensuring the delivery partner courier can time their arrival efficiently.
- For facilitating deliveries, there are a couple of options: either directly integrate order data with the delivery partner or establish a new 'delivery' data entity to forward delivery requests.
- As it allows for potential bundling of multiple orders and the flexibility to append more information if required.



A delivery payload would look like this :

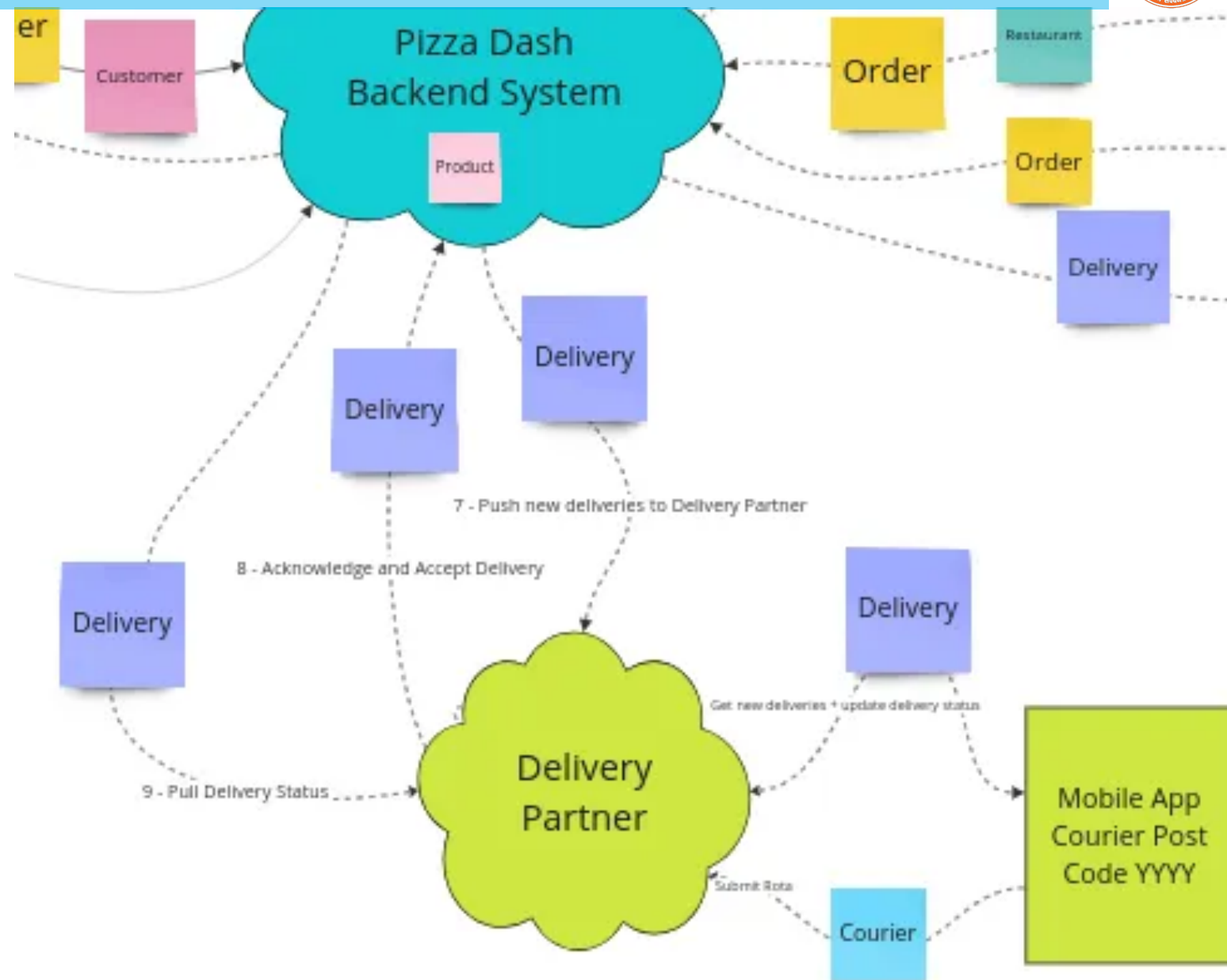
```
{
  "delivery": {
    "delivery": [
      {
        "delivery_num": "100554",
        "customer_first_name": "Peter",
        "phone_num": "4422522289988",
        "address": "11 Parker Street KK001",
        "courier_code": "FJ00026",
        "order": [
          {
            "order_num": "100050223",
            "order_datetime": "15-02-2026 14:02:01",
            "order_status": "Ready to deliver",
            "restaurant_num": "Pizza Dash London1"
          }
        ]
      }
    ]
  }
}
```

7 — Upon an order reaching the “Ready to Deliver” status, the backend system generates delivery requests to inform the delivery partner.

8 — The delivery partner’s system then assigns these orders to a suitable courier, based on predetermined delivery zones. Once a courier confirms the pickup, the delivery partner’s system updates with an acknowledgement of the accepted delivery.

9 — During the delivery process, the courier continually updates the delivery’s real-time location and status in the delivery partner’s system. Pizza Dash routinely checks this status. Eventually, when the courier completes the delivery, both the order and delivery statuses are updated to “Delivered”. Both the restaurant and the customer’s app are designed to track the order’s location and delivery status in real-time.

10 — After both the delivery and the order are marked as “Delivered”, the previously secured payment is processed and charged to the customer’s card. This action concludes the process for the customer, restaurant, and delivery partner.



This model highlights the primary data entities, their key fields, and the relationships.

Delivery	
Delivery Num*	Int - PK
Delivery Partner Num	varchar
Delivery Creation Timestamp	Timestamp
Order Number*	int - FK
Delivery Status*	varchar - lookup
Order Address*	address type
Customer First Name	varchar
Courier Num/Ref Num	varchar
Real Time Location	geoloc

Restaurant	
Restaurant Number*	Int - PK
Restaurant Name*	varchar
Address*	address type
Phone Number*	phone type
Business Hours	TBD

Customer	
Customer Num*	PK - varchar
First Name*	varchar
Last Name	varchar
Main Address	address type
Phone*	phone type
email address*	email type

Customer Preferences	
Preference ID*	Int - PK
Customer Num*	varchar - FK
Attribute Name	varchar
Attribute Value	varchar

We could store here Food Allergies if needed

Order	
Order Number*	Int - PK
Customer Num*	varchar - FK
Order Creation datetime*	datetime
Order Status*	varchar - lookup
Order Address*	address type
Restaurant Number	varchar - FK
Payment Method	varchar
Payment Status	varchar - lookup
Payment Confirm Ref	varchar

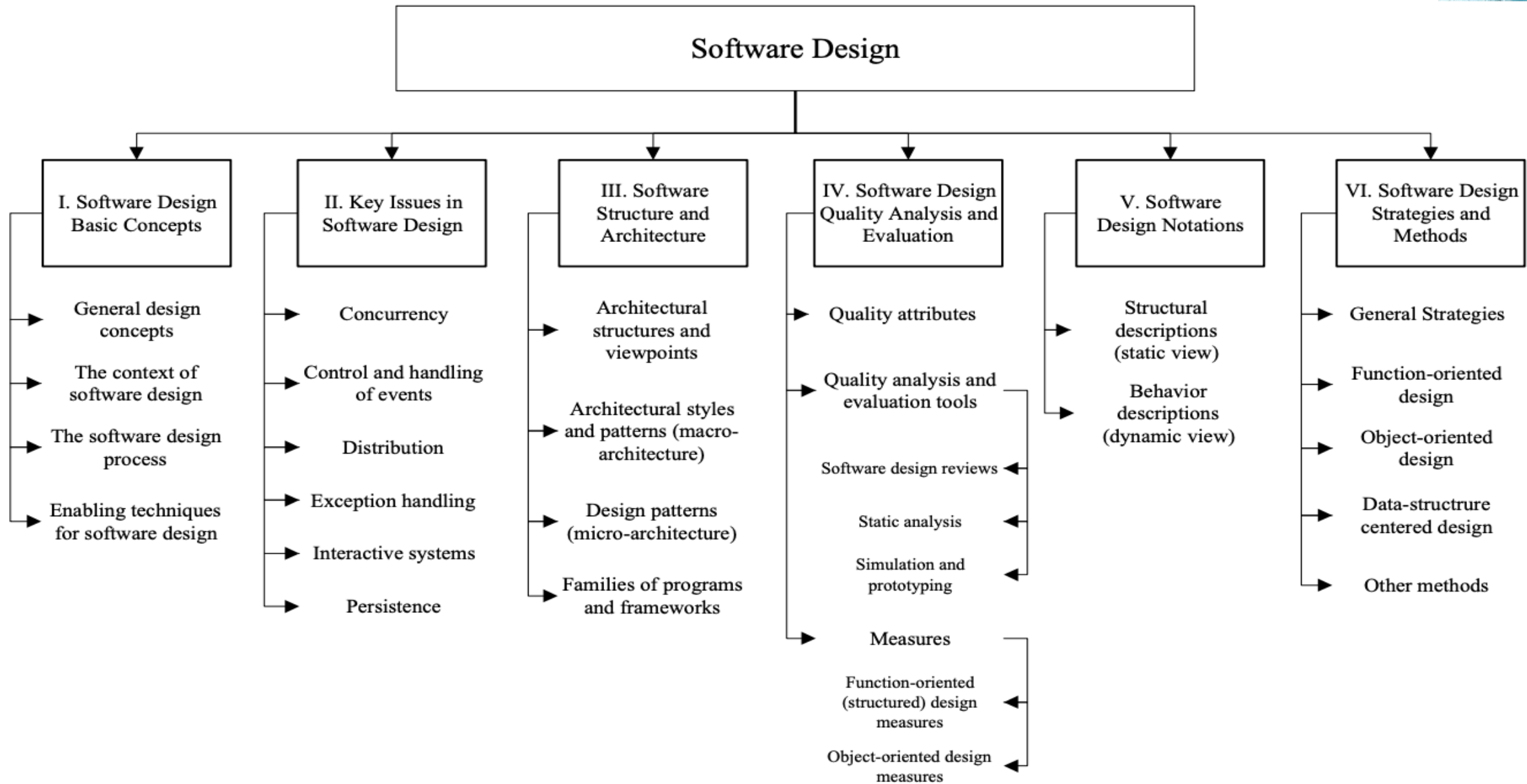
Order Events	
event id*	Int - PK
order number*	int - FK
event type*	varchar
event timestamp*	timestamp

Product	
Product Num*	varchar - PK
Prod Description*	varchar - text
Price*	Money
Picture	Blob

Order Detail	
Order Number*	Int - FK
Order Det Num*	Int - PK
Product Num*	varchar
Quantity*	int - float?
Price*	money
Prod Description	varchar - text

- The selected database technology should satisfy the following criteria to ensure optimal performance and security:
 - Data consistency and security
 - Scalability
 - High availability
 - Load management
- For the **mobile application**, choose a **NoSQL** database, such as Couchbase or something similar. These options are ideal for handling JSON format data and are known for their **scalability and availability**.
- When it comes to the **central system**, a NoSQL database offering **low latency** for both read and write operations could be effective. Choosing a distributed database like Apache Cassandra, known for its **replication** capabilities and **high failure tolerance**, might be a fit.
- A relational database could also be a consideration for the central system, especially if there's an anticipation of extensive reporting and analytics needs. However, this would largely hinge on the specific requirements for data analysis.

Software Design Breakdown



- A number of key issues must be dealt with when designing software systems. Some of these are really quality concerns that must be addressed by all systems, for example, **performance**.
- Another important issue is how to **decompose**, **organize** and **package the software components**. This is so fundamental that it must be addressed, in one way or another, by all approaches to design
- **Concurrency**: how to decompose the systems into processes, tasks and threads and deal with related efficiency, atomicity, synchronization and scheduling issues
- **Control and handling of events**: how to organize the flow of data and the flow of control, how to handle reactive and temporal events through various mechanisms, e.g., implicit invocation and call-backs

- **Distribution**: how the software is distributed on the hardware, how the components communicate, how middleware can be used to deal with heterogeneous systems
- **Error and exception handling and fault tolerance**: how to prevent and tolerate faults and deal with exceptional conditions
- **Interactive systems**: which approach to use to interact with users.
- **Persistence**: how long-lived data is to be handled.

- Different high-level facets of a software design can and should be described and documented.
- These facets are often called **views**: “a view represents a partial aspect of a software architecture that shows specific properties of a software system”.
- These different views pertain to different issues associated with the design of software.
- For example, the logical view (satisfying the functional requirements) vs. the process view (concurrency issues) vs. the physical view (distribution issues) vs. the development view (how the design is broken down into implementation units).
- Some Architects use different terminologies, e.g., behavioral vs. functional vs. structural vs. data modeling views.
- The key idea is that a software design is a multi-faceted artifact produced by the design process and generally composed of relatively independent and orthogonal views

- Architectural styles (macro-architectural patterns): An architectural style is “a set of constraints on an architecture [that] define a set or family of architectures that satisfy them”.
- An architectural style can thus be seen as a meta-model that can provide the high-level organization (the macro-architecture) of a software system.
- A number of major styles have been identified by various architects. These styles can (tentatively) be organized as follows
 - General structure (e.g., layers, pipes and filters, blackboard);
 - Distributed systems (e.g., client-server, three-tiers, broker);
 - Interactive systems (e.g., Model-View-Controller, Presentation-Abstraction-Control);
 - Adaptable systems (e.g., micro-kernel, reflection);
 - Other styles (e.g., batch, interpreters, process control, rule-based).

- Described succinctly, a pattern is “a common solution to a common problem in a given context”.
- Whereas architectural styles can be seen as patterns describing the high-level organization of software systems, thus their macro-architecture, other design patterns can be used to describe details at a lower, more local level, thus describing their microarchitecture.
- A wide range of patterns available, Such design patterns can (tentatively) be categorized as follows: -
 - Creational patterns: e.g., builder, factory, prototype, singleton.
 - Structural patterns: e.g., adapter, bridge, composite, decorator, façade, flyweight, proxy. -
 - Behavioral patterns: e.g., command, interpreter, iterator, mediator, memento, observer, state, strategy, template, visitor.

- Families of programs and frameworks: One possible approach to allow the reuse of software designs and components is to design families of systems – also known as software product lines – which can be done by identifying the commonalities among members of such families and by using reusable and customizable components to account for the variability among the various members of the family.
- In the field of OO programming, a key related notion is that of framework.
- A framework is a partially complete software subsystem which can be extended by appropriately instantiating some specific plug-ins (also known as hot spots).

What is Event-Driven Architecture?



- Event-driven architecture is a software design paradigm where the flow of the system is governed by event signals that indicate a change in state or the occurrence of an action.
- Unlike traditional architectures that often rely on direct calls between components, EDA decouples the communication between various parts of a system, enabling them to respond asynchronously to events as they happen.
- This decoupling is achieved through the use of event producers, event consumers, and event channels.
- Event producers generate events whenever something of significance occurs, such as a user action, a system update, or a sensor reading.
- These events are then transmitted over event channels, which can be message queues, event streams, or other forms of communication pathways.
- Event consumers, which are services or processes that listen for specific events, react accordingly by executing predefined actions, such as processing data, updating a database, or triggering further events.





Error handling in Event-Driven Architecture (EDA) is crucial for maintaining the reliability, stability, and performance of a system.

- **System Resilience:** Effective error handling ensures that the system can continue to operate smoothly even when unexpected issues arise. By managing errors gracefully, the system can isolate and contain problems, preventing them from affecting other components.
- **Data Integrity:** In EDA, data is often transmitted through events. Errors in event processing, if not handled correctly, can lead to incomplete or corrupted data. Robust error handling mechanisms, such as retry logic and dead-letter queues, help preserve data integrity by ensuring that events are either processed correctly or safely stored for later analysis.

- **Operational Visibility:** Proper error handling often involves logging and monitoring, which provide insights into the system's health. By tracking errors and their causes, developers can identify and address issues proactively, improving the overall reliability of the system.
- **Scalability and Performance:** As EDA systems scale, the likelihood of encountering errors increases. Without efficient error handling, the system might struggle to manage the growing volume of events, leading to performance degradation. Implementing strategies like circuit breakers and fallback mechanisms helps maintain system performance under heavy load or in the presence of persistent errors.
- **User Experience:** Errors that go unhandled can directly impact the end-user experience, leading to application crashes, slow response times, or inconsistent behavior. By catching and addressing errors promptly, EDA systems can deliver a more reliable and user-friendly experience.

In an Event-Driven System, various types of errors can occur, each with its own implications for system stability and performance. Understanding these errors is crucial for designing effective error-handling strategies. Here are some common types of errors encountered in event-driven systems:

1. Event Production Errors:

Data Validation Errors: Occur when the data used to generate an event is invalid or does not meet predefined criteria, such as missing fields or incorrect formats.

Timeouts: Happen when the event producer fails to generate an event within a specified time frame, often due to resource constraints or network delays.



2. Event Transmission Errors:

Failures: Occur when events cannot be transmitted due to network issues, leading to event loss or delays.

Message Queue Overflows: Happen when the event queue exceeds its capacity, causing events to be lost or delayed.

Serialization/Deserialization Errors: Occur when events cannot be properly serialized (converted to a transmittable format) or deserialized (converted back to a usable format) due to data corruption or incompatible formats.



3. Event Consumption Errors:

Processing Failures: Happen when the event consumer encounters an error while processing an event, such as a database write failure or an unhandled exception.

Concurrency Issues: Occur when multiple consumers attempt to process the same event simultaneously, leading to race conditions or deadlocks.

Resource Limitations: Happen when event consumers run out of resources (e.g., memory, CPU) needed to process events, leading to crashes or degraded performance.

4. System-Level Errors:

Dependency Failures: Occur when external systems or services that the event-driven system depends on fail, leading to unprocessed or delayed events.

Configuration Errors: Happen when incorrect or inconsistent configurations cause components to behave unexpectedly, leading to errors in event handling or routing.

Security Issues: Include unauthorized access or tampering with events, which can lead to data breaches or compromised system integrity.

5. Logical Errors:

Business Logic Failures: Occur when the event handling logic does not align with the intended business rules, leading to incorrect or unexpected outcomes.

Event Looping: Happen when events trigger a loop of actions that unintentionally generate more events, causing infinite loops or resource exhaustion.

6. Event Ordering Errors:

Out-of-Order Events: Occur when events are processed in the wrong sequence, leading to inconsistent state changes or data corruption.

Duplicate Events: Happen when the same event is processed multiple times, potentially leading to redundant or conflicting actions.

- Software analysis and design tools are essential for developing high-quality software systems. They assist in understanding, modeling, and planning software projects, ensuring that the final product meets user needs and performs reliably.
- These tools facilitate various stages of the software development lifecycle, including requirements gathering, system design, and architectural planning.
- Software analysis and design includes all activities, which help the transformation of requirement specification into implementation. Requirement specifications specify all functional and non-functional expectations from the software.
- These requirement specifications come in the shape of human readable and understandable documents, to which a computer has nothing to do.
- Software analysis and design is the intermediate stage, which helps human-readable requirements to be transformed into actual code.

Software analysis and design tools are specialized applications used to support various stages of the software development lifecycle. They help developers and designers understand, plan, model, and document software systems to ensure that the final product meets requirements and functions effectively.

- **Requirement Gathering:** Tools help capture and document user requirements, ensuring all stakeholder needs are considered and addressed.
- **Modeling:** They enable the creation of visual models, such as flowcharts, UML diagrams, and data models, to represent the system's structure and behavior.
- **Design:** Tools assist in designing the software architecture and detailed design, including user interfaces, data storage, and system interactions.
- **Documentation:** They help in generating and maintaining comprehensive documentation, which is crucial for communication among team members and future maintenance.
- **Analysis:** Tools provide capabilities to analyze software designs for potential issues, such as performance bottlenecks or design flaws, ensuring the robustness of the final product.
- **Collaboration:** They support collaborative work by allowing multiple team members to work together on the same project, facilitating coordination and integration.



Check the following link for Software Architecture Documentation related data -

<https://document360.com/blog/software-architecture-documentation/>



Thank You !
for Learning
Software Architecture Analysis